

Softverski algoritmi u sistemima automatskog upravljanja

Uvod u mašinsko učenje

Emina Demirović

Katedra za automatsko upravljanje
Departman za računarstvo i automatiku
Fakultet tehničkih nauka

Način polaganja

Teorija	10 bodova
Projekat - kod i dokumentacija	20 bodova
Odbrana projekta	10 bodova

Termini vežbi: 04. 05. - 16. 06. 2026.

Termini polaganja: 22. - 27. 06. 2026.

Emina Demirović

NTP-416

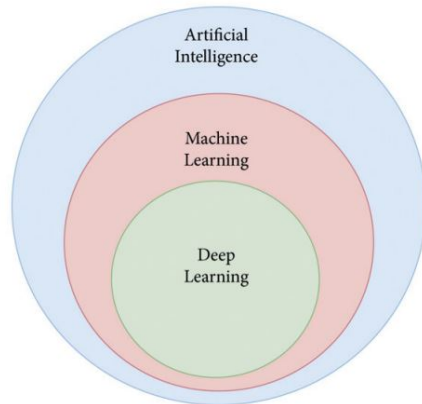
demirovic@uns.ac.rs

Sadržaj termina

- ▶ mašinsko učenje (u kontekstu veštačke inteligencije)
- ▶ osnovna ideja ML-a i važni pojmovi
- ▶ osnovne vrste mašinskog učenja
- ▶ primer: predviđanje cene stana
- ▶ regresija i klasifikacija
- ▶ ML pipeline: od podataka do rezultata
- ▶ train, validation i test skupovi
- ▶ data leakage, baseline model i evaluacija
- ▶ osnovni workflow i organizacija ML projekta

Mašinsko učenje u okviru VI

- ▶ **Veštačka inteligencija** - pokušaj da se proces učenja i inteligencije prenese na računare kroz razvoj sistema sposobnih da obavljaju zadatke za koje je inače potreban čovek.
- ▶ **Mašinsko učenje** - podoblast veštačke inteligencije koja se bavi algoritmima za učenje iz podataka.
- ▶ **Duboko učenje** - uža oblast ML-a zasnovana na dubokim neuronskim mrežama.



Osnovna ideja mašinskog učenja

- ▶ nije potrebno (nekada teško/nemoguće) pisati sva pravila ručno
- ▶ model dobija podatke
- ▶ tokom obučavanja pokušava da nauči obrazac
- ▶ naučeni obrazac koristi za nove primere
- ▶ modeli služe za predikcije, preporuke, detekciju obrazaca i donošenje odluka

Model ne dobija savršeno pravilo, već uči približnu vezu između ulaza i izlaza.

Šta je mašinsko učenje?

Mašinsko učenje je proces pronalaženja algoritama koji automatski mapiraju ulazne podatke u korisne izlaze, učeći direktno iz primera umesto iz ljudskih instrukcija.

- ▶ **podaci** predstavljaju iskustvo iz kog model uči
- ▶ **model** je naučena veza između ulaza i izlaza
- ▶ **rezultat** je izlaz modela za novi primer

Formalna definicija ML

Mašinsko učenje

Za računarski program se kaže da uči iz iskustva E , u odnosu na zadatak T i meru performansi P , ako se njegove performanse na zadatku T , merene sa P , poboljšavaju sa iskustvom E .

- ▶ E : iskustvo, odnosno podaci
- ▶ T : zadatak koji model treba da reši
- ▶ P : mera uspešnosti modela

Kada ML ima smisla?

ML ima smisla kada imamo:

- ▶ dovoljno podataka
- ▶ obrazac koji se može naučiti
- ▶ jasno definisan cilj
- ▶ način da izmerimo uspeh
- ▶ potrebu da radimo predikcije nad novim primerima

Bez podataka, cilja i mere uspeha nema ozbiljnog ML problema.

Ako je pravilo jednostavno, poznato i stabilno, ML nam najčešće nije potreban.

Primer: predviđanje cene nekretnina

Problem

Proceniti cenu nekretnine na osnovu njenih osobina.

- ▶ osobine
- ▶ očekivani izlaz
- ▶ skup podataka

Ideja

osobine stana $\rightarrow$ cena stana

Osnovni pojmovi

Dataset

Skup podataka koji koristimo za obučavanje, validaciju i testiranje modela.

- ▶ **uzorak**: jedan primer u skupu podataka
- ▶ **atribut**: osobina koju opisujemo u podacima
- ▶ **feature**: ulazni atribut koji model koristi
- ▶ **label** ili **target**: vrednost koju želimo da predvidimo

Model, predikcija i greška

- ▶ **model**: naučena veza između ulaznih atributa i ciljne vrednosti
- ▶ **predikcija**: procena koju model daje za novi primer
- ▶ **greška**: razlika između stvarne i predviđene vrednosti

Primer

Stvarna cena: 100 000

Predviđena cena: 108 000

Greška: 8 000

Osnovne vrste mašinskog učenja

- ▶ **nadgledano učenje:** podaci imaju poznate izlaze
- ▶ **nenadgledano učenje:** tražimo strukturu bez poznatih izlaza
- ▶ **polunadgledano učenje:** deo podataka ima oznake, deo nema
- ▶ **učenje potkrepljenjem:** agent uči kroz nagrade i kazne

Nadgledano učenje

Ideja

Nadgledano učenje predstavlja proces kreiranja modela na osnovu skupa podataka koji sadrži i ulazne podatke i odgovarajuće željene izlaze (oznake).

- ▶ ulaz: osobine stana
- ▶ poznat izlaz: stvarna cena stana
- ▶ cilj: predvideti cenu za novi stan

Cilj algoritama SL jeste da nauče vezu između ulaza i izlaza, kako bi mogli da predviđaju ili klasifikuju novovidećene podatke.

Ovaj pristup je osnova za mnoge praktične primene kao što su prepoznavanje slika, klasifikacija teksta, predviđanje vremenskih serija i druge.

Regresija

Regresija

Regresija je tehnika koja se koristi za modelovanje i opisivanje veze između ulaznih podataka i očekivanog izlaza kada je cilj predviđanje kontinualnih vrednosti.

- ▶ temperatura
- ▶ potrošnja energije
- ▶ vreme isporuke

Veza iz koje model uči se izražava matematičkom jednačinom koja omogućava kvantitativno predviđanje rezultata na osnovu poznatih ulaza. Konačni rezultat regresione analize je takođe brojna vrednost

Klasifikacija

Klasifikacija

Klasifikacija je proces razvrstavanja podataka u unapred definisane kategorije, sa ciljem da model na osnovu poznatih ulaznih obeležja pravilno predvidi kojoj kategoriji novi podatak pripada.

- ▶ spam ili nije spam
- ▶ kvar ili normalan rad
- ▶ odobren ili odbijen zahtev
- ▶ tip objekta na slici

Ovaj pristup se koristi kada je izlaz **diskretna** kategorija

Tokom procesa obuke, klasifikator uči iz obeleženih podataka kako bi pronašao granice koje razdvajaju različite klase.

Primeri algoritama

Regresija

- ▶ linearna regresija
- ▶ ridge regresija
- ▶ lasso regresija
- ▶ SVM za regresiju

Klasifikacija

- ▶ logistička regresija
- ▶ stablo odluke
- ▶ Naive Bayes
- ▶ K-najbližih suseda
- ▶ SVM za klasifikaciju

Najvažnije je pravilno formulisati problem!

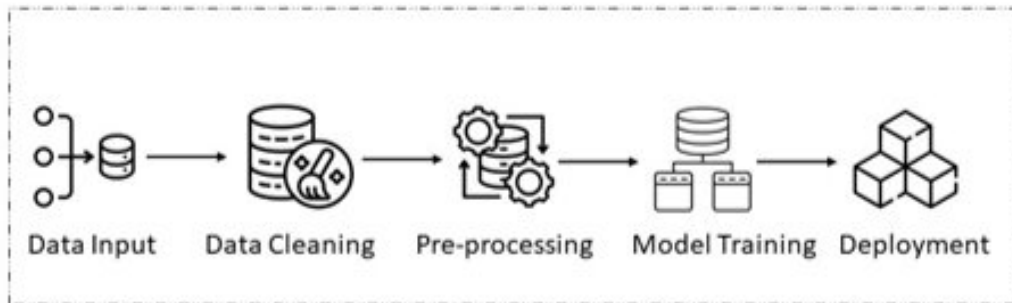
Od realnog problema do ML zadatka

- ▶ realni problem: proceniti cenu stana
- ▶ ulaz: osobine stana
- ▶ izlaz: cena stana
- ▶ tip problema: regresija
- ▶ mera uspeha: greška procene

ML pipeline

Uređen tok koraka kroz koje prolazimo od realnog problema i podataka do modela, rezultata i zaključka.

Osnovni ML pipeline



Prikupljanje i priprema podataka

Model uči samo iz informacija dostupnih u dataset-u.

Data Wrangling

Proces sređivanja podataka tako da budu upotrebljivi za analizu i model.

Potencijalni problemi i rešenja:

- ▶ nedostajuće vrednosti → obrada
- ▶ anomalije → analiza i uklanjanje
- ▶ duplikati → uklanjanje
- ▶ ekstremne vrednosti → normalizacija i skaliranje
- ▶ nekonzistentne kategorije → encoding

Eksplorativna analiza podataka

Eksplorativna analiza podataka je značajna za bolje razumevanje podataka, a samim tim i problema.

EDA

Cilj je ustanoviti da li postoje šabloni koji se javljaju u skupu - zavisnosti između različitih atributa i da li se atributi mogu nekako „povezati“ u cilju uprošćavanja skupa podataka.

Tehnike analize:

- ▶ vizualizacija podataka
- ▶ prikaz raspodele (varijansa, medijana, srednja vrednost)
- ▶ pronalaženje eventualne korelacije između atributa
- ▶ redukcija dimenzionalnosti

Odabir pogodnog algoritma

Izbor algoritma zavisi od više faktora:

- ▶ tip problema: regresija ili klasifikacija
- ▶ veličina skupa podataka
- ▶ broj atributa
- ▶ interpretabilnost modela
- ▶ očekivane performanse
- ▶ složenost modela

Train, validation i test skup

- ▶ **train:** model uči parametre
- ▶ **validation:** odabir modela i podešavanje hiperparametara
- ▶ **test:** konačna provera - evaluacija

Primer dobre prakse:

70/15/15 ili 80/10/10.

Napomene:

Dobar rezultat na trening podacima ne znači nužno dobar model.

Test skup se ne koristi za donošenje odluka tokom razvoja modela.

Unakrsna validacija

Unakrsna validacija je tehnika procene performansi modela koja se zasniva na podeli dostupnih podataka u više manjih podskupova.

Cross-validation

Cilj je da se model trenira i testira više puta na različitim delovima podataka kako bi se osiguralo da su njegovi rezultati pouzdani i da se on može uspešno primeniti na novoviđene podatke.

Ideja:

- ▶ dataset se podeli na više delova
- ▶ model se trenira više puta
- ▶ svaki deo jednom služi kao validacioni deo
- ▶ posmatra se prosečan rezultat

Curenje podataka

Data leakage

Situacija u kojoj informacije koje ne bi smele biti dostupne modelu procure u trening ili evaluaciju, te zbog koje model daje lažno dobre rezultate.

Problemi:

- ▶ rezultati deluju bolje nego što zaista jesu
- ▶ model ne uči opšte pravilo, već koristi nedozvoljenu informaciju
- ▶ greška često nije očigledna u samom kodu

Rešenje

Podeliti podatke pa tek onda uraditi preprocessing na train skupu i primeniti ga na validation/test skup.

Baseline model

Baseline

Jednostavan početni model ili pravilo sa kojim se porede ozbiljniji modeli.

- ▶ služi kao početna referentna tačka
- ▶ pokazuje minimalni rezultat koji treba pobediti
- ▶ mora biti jasan i lako proverljiv

Ako složeniji model nije bolji od baseline-a, rešenje sigurno nije dobro.

Treniranje modela

- ▶ model dobija train skup
- ▶ pokušava da nauči vezu između atributa i cene
- ▶ tokom treninga se podešavaju parametri modela

Evaluacija regresionog modela

koliko su predikcije udaljene od stvarnih brojčanih vrednosti - primeri metrika: MAE, MSE, RMSE, R^2 score ...

Evaluacija klasifikacionog modela

koliko dobro model razvrstava primere u odgovarajuće klase - primeri metrika: tačnost, preciznost, odziv, F1 score ...

ML projekti se razvijaju **iterativno!**

Napomena: Model se ne ocenjuje po složenosti, već po sposobnosti generalizacije.

Underfitting i overfitting

Underfitting

Model je previše jednostavan - slabo se prilagođava obučavajućem skupu podataka i ne uspeva da zaključi kako formirati izlaz.

- ▶ loš rezultat na train skupu
- ▶ loš rezultat na validation skupu

Rešenje

- ▶ promeniti algoritam
- ▶ proširiti skup podataka

Overfitting

Model se previše dobro prilagođava obučavajućem skupu podataka.

- ▶ dobar rezultat na train skupu
- ▶ loš rezultat na validation/test skupu

Rešenje

- ▶ proširiti skup podataka
- ▶ regularizacija
- ▶ ansambl metode

Regularizacija

Regularizacija

Regularizacija je tehnika u mašinskom učenju kojom se u funkciju greške dodaje kazneni član, sa ciljem da se model obeshrabri u učenju previše kompleksnih šablona i da se spreči ekstremno visoka vrednost njegovih parametara.

- ▶ smanjuje rizik od preprilagođavanja
- ▶ pomaže kod multikolinearnosti
- ▶ stabilizuje model (mala varijansa)

Ukupni gubitak = greška predviđanja + kazna za kompleksnost

Radno okruženje i alati

Za vežbe

- ▶ programski jezik **Python**
- ▶ razvojno okruženje po izboru - **PyCharm** / **Visual Studio Code**
- ▶ **Google Colab** za interaktivan rad
- ▶ osnovne biblioteke za rad sa podacima: **NumPy**, **pandas**, **matplotlib**
- ▶ klasični ML modeli, preprocessing i metrike: **scikit-learn**
- ▶ dataset-ovi - ugrađene biblioteke ili internet (Kaggle)

Za projekat

- ▶ predaja i verzionisanje projekta: **GitHub**
- ▶ UI ili API za model - **Streamlit** / **FastAPI**
- ▶ **Docker**

Predlog organizacije projekta

```
ml-project/  
  data/  
    raw/  
    processed/  
  src/  
    data_preparation.py  
    train.py  
    evaluate.py  
    predict.py  
  models/  
  results/  
    figures/  
    metrics/  
  app/  
  README.md  
  requirements.txt
```

Šta nas očekuje u nastavku semestra?

- ▶ Linearni modeli za regresiju i klasifikaciju
- ▶ Evaluacija modela i generalizacija
- ▶ Metode zasnovane na rastojanju i sličnosti
- ▶ Stabla odlučivanja i ansambl metode
- ▶ Eksportovanje i čuvanje modela - deployment
- ▶ Primena ML u praksi
- ▶ Primer ML istraživačkog rada
- ▶ Dobre prakse u razvoju ML rešenja